

Essay 1c: From Bandits to Trees

The algorithms in Essay 1b — TD learning, Q-learning, policy gradients — all share a common limitation: they react to states they have seen, but they do not search forward through states they haven't. This essay introduces the key idea that bridges value learning and deliberate search: the explore/exploit trade-off. We start at a slot machine, work through the mathematics of why exploration cannot be avoided, and arrive at the exact optimal solution — the Gittins index — whose structure illuminates why the discount factor γ is not merely convenient but necessary.

1. The Multi-Armed Bandit

Imagine you are standing in front of a row of slot machines. Each machine has a different payout rate, but you don't know which is which. Pull a machine, collect (or lose) some coins, and repeat. Your goal is to maximise your total winnings over some fixed number of pulls.

This is the multi-armed bandit problem — “multi-armed” because each machine is like a different arm of the same bandit. The name is old and slightly tongue-in-cheek, but the problem it describes is ubiquitous: it appears in clinical trials (which drug to administer?), online advertising (which ad to show?), and — as we will see — game tree search.

The core tension is between *exploration* and *exploitation*. Exploitation means pulling the machine that has paid off best so far. Exploration means trying machines you haven't pulled much, because one of them might be better than your current best.

Brian Christian and Tom Griffiths, in *Algorithms to Live By*, put it sharply: “The value of exploration can only go down over time.” Early in your time at the casino, information about unknown machines is enormously valuable — you have many future pulls in which to benefit from it. Late in your time, exploration is wasteful: you will not live long enough to recover the cost of a bad pull. The optimal strategy is front-loaded exploration that tapers into pure exploitation as the horizon approaches.

This insight has a precise mathematical form. The *regret* of a strategy is the difference between what you earned and what you would have earned if you had known the best machine from the start and pulled it every time:

$$R(N) = N \cdot \mu^* - \sum_{t=1}^N r_t$$

where μ^* is the expected payout of the best machine and r_t is your reward on pull t . We can rewrite this in terms of suboptimal arm pulls:

$$R(N) = \sum_{i: \mu_i < \mu^*} (\mu^* - \mu_i) \cdot E[T_i(N)]$$

where $\Delta_i = \mu^* - \mu_i$ is the *suboptimality gap* of arm i , and $T_i(N)$ is the number of times arm i is pulled in N rounds. Minimising regret means pulling suboptimal arms as rarely as possible — but “as rarely as possible” turns out to be more than you might hope.

2. Why Logarithmic Regret is Optimal: The Lai-Robbins Bound

The remarkable result from bandit theory is that no strategy, however clever, can achieve regret better than $O(\log N)$. This is not a limitation of UCB — it is a mathematical ceiling shared by every consistent algorithm.

The argument is information-theoretic. To avoid wasting pulls on arm i , you must be confident that arm i is suboptimal. But how can you become confident of this? Only by pulling arm i enough times to statistically distinguish its true mean μ_i from the best arm’s mean μ^* . And this takes a number of samples that grows with N .

Formally, consider two variants of the problem: in version A, arm i has mean μ_i ; in version B, arm i has mean μ^* (making it the best arm). Your strategy cannot inspect the problem description — it can only observe rewards. To pull arm i the right number of times in each version, it must statistically distinguish A from B by looking at the rewards from arm i alone.

The Kullback-Leibler (KL) divergence $KL(\mu_i \parallel \mu)$ *measures how distinguishable these two hypotheses are per sample: a large KL means the arm’s payouts look quite different under μ_i versus μ , so few samples suffice; a small KL means the distributions are nearly identical, requiring many samples to tell them apart.*

Lai and Robbins formalised this in 1985:

Theorem (Lai-Robbins, 1985). For any *consistent* strategy — one whose regret is $o(N^\alpha)$ for every $\alpha > 0$ — and for any arm i with $\mu_i < \mu^*$:

$$E[T_i(N)] \geq (1 + o(1)) \cdot \log N / KL(\mu_i \parallel \mu^*)$$

Summing over all suboptimal arms, any consistent strategy must suffer regret at least:

$$\liminf_{N \rightarrow \infty} R(N) / \log N \geq \sum_{i: \mu_i < \mu^*} (\mu^* - \mu_i) / KL(\mu_i \parallel \mu^*)$$

This is not a statement about bad algorithms. It is a statement about the structure of

the problem: to learn that arm i is suboptimal, you must try it logarithmically often in N . There is no shortcut.

The intuition is clean. Suppose you have tried arm i only k times. With k samples from a distribution with mean μ_i , you can reject the hypothesis “this arm has mean μ ” with probability roughly $1 - \exp(-k \cdot KL(\mu_i \parallel \mu))$. For this probability to approach 1 as N grows, you need k to grow as $\log N / KL(\mu_i \parallel \mu^*)$. Below this threshold, there is a non-negligible chance that arm i is actually the best arm, which any rational strategy must account for by continuing to try it.

The UCB Formula

UCB works by maintaining, for each arm a , two quantities: the empirical mean reward $\hat{\mu}(a)$ — the average payout observed so far — and a confidence bonus that quantifies how uncertain we are about that estimate.

The UCB score for arm a at time t is:

$$UCB(a) = \hat{\mu}(a) + c \cdot \sqrt{(\log t / n(a))}$$

where $n(a)$ is the number of times arm a has been pulled, and c is a constant controlling the balance between exploration and exploitation.

The first term, $\hat{\mu}(a)$, is exploitation: favour arms that have paid off well. The second term, $\sqrt{(\log t / n(a))}$, is exploration: favour arms that have been pulled less often (small $n(a)$) relative to total pulls (large t).

The key insight — which Christian and Griffiths call “optimism in the face of uncertainty” — is that UCB always picks the arm with the highest *upper confidence bound*: the best plausible payout, not just the best observed payout. By acting optimistically about uncertain arms, UCB is forced to explore them; once an arm has been tried enough times, its uncertainty collapses and the empirical mean dominates.

UCB achieves the Lai-Robbins lower bound up to constant factors. Auer, Cesa-Bianchi, and Fischer (2002) proved that UCB1 satisfies:

$$E[R(N)] \leq \sum_{i: \mu_i < \mu^*} (8 \log N / \Delta_i + (1 + \pi^2/3) \cdot \Delta_i)$$

The leading term, $8 \log N / \Delta_i$, matches the Lai-Robbins lower bound up to the constant factor and the replacement $KL(\mu_i \parallel \mu^*) \rightarrow \Delta_i^2 / 2$. UCB thus pulls each suboptimal arm exactly as often as the information-theoretic argument demands — no more, no less.

3. The Gittins Index: The Exact Optimal Solution

UCB achieves logarithmic regret, but for the discounted bandit — where the total objective is $\sum \gamma^t r_t$ rather than total reward — there is a celebrated exact solution that predates UCB by nearly two decades.

John Gittins proved in 1979 that the optimal policy for K independent arms under geometric discounting has a remarkably simple structure: at each step, play the arm with the highest *Gittins index* $g(s_i)$, where s_i is the current state of arm i .

The Gittins index $g(s)$ of an arm in state s is defined as:

$$g(s) = \sup_{\tau \geq 1} E_s[\sum_{t=0}^{\tau-1} \gamma^t r_t] / E_s[\sum_{t=0}^{\tau-1} \gamma^t]$$

This is the supremum over stopping times τ of the ratio: expected discounted reward collected while playing this arm, divided by expected discounted time spent playing it. It is the arm's "fair rate" of return — equivalently, the fixed per-period reward λ^* that would make you indifferent between "play this arm" and "collect λ^* each period and stop."

Christian and Griffiths describe the Gittins index as the mathematically exact answer to the explore/exploit problem: a single number that perfectly summarises an arm's future prospects, reducing the K -arm problem to K independent single-arm problems.

Why $\gamma < 1$ is necessary — three interlocking reasons

The Gittins index requires $\gamma < 1$ not as a convenience but as a structural necessity. The discount factor does three things simultaneously, and all three are load-bearing.

First: the index must be finite. The definition involves the ratio of two infinite sums. When $\gamma < 1$, both the numerator $E_s[\sum \gamma^t r_t]$ and the denominator $E_s[\sum \gamma^t] = 1/(1-\gamma)$ converge to finite values, and the supremum over τ is well-defined. When $\gamma = 1$, both sums diverge, the ratio is of the form ∞/∞ , and the index is undefined. No Gittins index exists without discounting.

Second: the index is computed by solving a Bellman equation. Given a trial value λ , define the "excess value" of arm i in state s :

$$V(s, \lambda) = \max \{ 0, E_s[r - \lambda + \gamma \cdot V(s', \lambda)] \}$$

Here $V(s, \lambda)$ measures how much better it is to play the arm than to collect λ each period. The Gittins index $g(s)$ is the unique λ^* at which $V(s, \lambda^*) = 0$ — the breakeven rate. This is a fixed-point equation for V , and it has a unique solution precisely because $\gamma < 1$ makes the backup operator a contraction: for any two functions V and U ,

$$\|T_\gamma V - T_\gamma U\|_\infty \leq \gamma \cdot \|V - U\|_\infty$$

By the Banach fixed-point theorem, there is exactly one solution. Without $\gamma < 1$, there is no contraction, no fixed-point guarantee, and the computation of the Gittins index breaks down.

Third: the decomposition theorem requires finite effective horizons. Gittins’ proof that playing $\arg\max_i g(s_i)$ is globally optimal relies on the arms being *independent* and on the discount factor making the future worth less than the present. Intuitively: with $\gamma < 1$, each arm contributes a geometrically shrinking stream of future rewards, and this shrinkage allows the joint optimisation over all K arms to decompose into K separate one-dimensional problems. The mathematical mechanism is a “retirement reward” argument: each arm can be replaced by a hypothetical option to retire it at any time and collect a fixed annuity, and the optimal retirement annuity is exactly the Gittins index. This decomposition works because the geometric discounting makes each arm’s contribution to future play separable from the others’. When $\gamma = 1$, the arms interact through a shared constraint on total plays, the decomposition fails, and no simple index characterises the optimal policy.

The Gittins index thus provides a deeper account of why $\gamma < 1$ matters — one that goes beyond “the Bellman equation needs a contraction.” The discount factor is the key that unlocks a beautiful reduction: an intractable K -dimensional optimisation collapses into K independent Bellman equations, each with a unique solution, because the geometric structure of discounting makes the arms separable. Remove $\gamma < 1$, and both the reduction and the uniqueness disappear simultaneously.

4. From Bandits to Trees: PUCT

Now take the bandit problem and put it inside a game tree.

At every node in the tree, we face a choice of which branch to explore next. This is exactly a bandit problem: the “arms” are the child nodes (the available moves), and the “payout” of pulling an arm is the value of the game continuation that arm leads to.

AlphaZero uses a version of UCB called PUCT (Predictor + Upper Confidence bound applied to Trees):

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \sqrt{(\sum_b N(s, b)) / (1 + N(s, a))}$$

Breaking this down:

- $Q(s, a)$ is the exploitation term: the empirical mean value of taking action a from state s , averaged over all simulated game continuations that have passed through this node.
- $N(s, a)$ is the visit count for (s, a) : how many simulations have explored this branch.
- $\sum_b N(s, b)$ is the total visit count for state s across all actions: a proxy for total time.
- $P(s, a)$ is the *prior probability* of action a from state s — a prediction from the neural network about how promising this move looks, before any search. This is the “Predictor” part.

The PUCT formula does to the game tree what UCB does to the slot machines: it biases exploration toward promising, undersampled branches while preventing any one branch from being completely ignored. The prior $P(s, a)$ acts as an intelligent initialisation — instead of treating all arms as equally unknown, we start with the neural network’s best guess about which actions are most worth exploring.

Note how the chain from raw moves to search scores works. The neural network outputs two things for each position: a *value head* $V_\theta(s)$ estimating the win probability from that state, and a *policy head* giving $P(s, a)$ for each legal action. During search, $Q(s, a)$ is updated by averaging the V_θ values from all leaf nodes reached via that branch. So the chain is: raw board position $\rightarrow$ neural network $\rightarrow (V_\theta, P) \rightarrow$ PUCT scores $\rightarrow$ which branch to expand next $\rightarrow$ richer Q estimates $\rightarrow$ better PUCT scores.

The prior $P(s, a)$ is not fixed. It is learned — initially random, then trained on the tree search outputs, which are themselves improved by the neural network. This feedback loop is AlphaZero’s core: search improves policy, policy improves search.

5. Building the Tree

Understanding PUCT for a single node is the first step. Understanding how it builds a tree across thousands of simulations is the second.

Each Monte Carlo Tree Search simulation follows four phases:

Selection: Starting from the root (the current game position), repeatedly select the child with the highest PUCT score until reaching a node that has not yet been expanded (a “leaf” of the current tree).

Expansion: Add the leaf node to the tree. The neural network evaluates this position, producing $V_\theta(s)$ and $P(s, a)$ for all legal actions.

Backup: Propagate $V_\theta(s)$ back up the tree, updating $Q(s', a')$ at every ancestor node along the path. Visit counts $N(s', a')$ are also incremented.

Repeat: Return to Selection for the next simulation.

After thousands of simulations, the tree has a rich picture of which branches are worth exploring. The PUCT formula has been ensuring throughout that high-prior, high-value branches are explored intensively, while low-prior, low-value branches are not entirely abandoned.

At the end of search, the agent plays the action with the highest visit count $N(s, a)$ from the root — not the highest $Q(s, a)$. Visit counts are a more robust statistic than Q -values: a branch visited many times has many independent estimates backing its value, while a branch visited once might have gotten lucky on a single V_θ evaluation.

6. What This Changes About Credit Assignment

Recall the credit-assignment problem from Essay 1b. The challenge was: a single game produces a single outcome, and we must attribute that outcome to forty individual decisions.

MCTS changes the character of this problem in a subtle but important way. Instead of learning from full game outcomes, AlphaZero’s neural network learns to predict the outcomes of *MCTS searches*. The training target for the policy head is not “did this game end in a win?” — it is “what fraction of MCTS simulations visited each action?” The training target for the value head is not “what was the final game outcome?” — it is the mixture of the MCTS-backed value estimate and the actual game result.

MCTS compresses the credit-assignment problem. A raw game outcome rewards or penalises every decision equally. An MCTS search outcome is local: it says something specific about the position the search was run from. By running many searches — at every move of every training game — and training the network on the resulting targets, AlphaZero sidesteps the worst of the credit-assignment problem without eliminating it entirely.

The residual credit-assignment problem — how to update the neural network weights efficiently — is handled by standard policy gradient and value regression techniques, which we covered in Essay 1b.

7. Why UCB Matters Here

It might seem that UCB and bandits are just an analogy — a pleasant framing for what is really just a search heuristic. But the connection runs deeper.

The Lai-Robbins lower bound guarantees that any consistent search strategy must explore each branch at least logarithmically often relative to the most explored branch. MCTS, by using PUCT (a form of UCB), meets this bound. In a game with a decisive tactical line hiding several moves deep, this is what ensures the tree search will eventually find it — no consistently good branch will be permanently overlooked.

Without a principled explore/exploit strategy, naive tree search either explores too broadly (wasting simulations on obviously bad moves) or too narrowly (committing too early to a seemingly good line and missing decisive alternatives). UCB’s logarithmic regret bound is the guarantee that neither failure mode dominates.

The prior $P(s, a)$ from the neural network reduces the constant factor in the logarithm. A strong prior concentrates search on moves that are actually worth exploring, reducing the effective branching factor and allowing deeper search in the time available. A weak prior — as at the start of training, when the network is random — produces near-uniform exploration, which is slow but still eventually correct.

This is the recursive beauty at the heart of AlphaZero: the Lai-Robbins argument guarantees the search will find the right lines; the Gittins insight guarantees the discount factor makes the value estimates well-defined and unique; and the neural network, by learning from search outcomes, progressively concentrates exploration exactly where Lai-Robbins demands it.

Next: Essay 2 dives into the full AlphaZero MCTS loop — self-play, tree search, and the training pipeline — and shows how these pieces fit together into a system that starts from random play and converges to superhuman performance.

Code: [Notebook 1 — The UTTT Game Engine](#) builds the game engine that MCTS runs on — legal move generation, immutable state transitions, and the random-agent baseline whose limitations motivate everything in this essay.